

Utilizando *while*

O `while` repete **enquanto** uma condição for verdadeira, sem saber de antemão quantas voltas vai dar. É o laço certo para pedir de novo até a resposta ser válida, para menus e para "até o usuário teclar Enter". Nesta aula: contador, validação de entrada, sentinela, `while True` com `break`, o `else` do laço e o laço infinito.

`while` condição

Contador

Validar entrada

Sentinela

`while True + break`

`while ... else`

Ctrl + C

Formação Claude Code 2026 · Baseado em docs.python.org (Reference: The while statement; Tutorial: break and continue, else Clauses on Loops), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/`, com as respostas dos arquivos `.in` · Saídas reais

Enquanto *for verdade*

contador.py

```
contador = 1
while contador <= 5:
    print("Volta", contador)
    contador += 1      # sem isso, o loop nunca termina
print("Fim, contador =", contador)
```

SAÍDA

```
Volta 1
Volta 2
Volta 3
Volta 4
Volta 5
Fim, contador = 6
```

- ✓ Antes de cada volta, o Python **testa a condição**
- ✓ Verdadeira: roda o bloco e testa de novo. Falsa: segue depois do laço
- ✓ Algo dentro do bloco precisa **mudar** a condição, senão nunca termina
- ✓ O contador termina em 6: foi o valor que fez a condição ficar falsa

Para "repita 5 vezes", o `for i in range(5)` (aula 14) é mais simples. O `while` brilha quando não se sabe quantas vezes.

Perguntar *até acertar*

validar.py

```
idade = input("Idade: ")
while not idade.isdigit():
    print("Digite só números.")
    idade = input("Idade: ")
print("Idade válida:", int(idade))
```

SAÍDA (DIGITOU "VINTE", "20" E 29)

```
Idade: vinte
Digite só números.
Idade: 20
Digite só números.
Idade: 29
Idade válida: 29
```

- ✓ Resolve o **ValueError** da aula 04: só converte quando já sabe que é número
- ✓ `.isdigit()` (aula 08) é a condição
- ✓ "20" tem a letra O no lugar do zero: o laço pegou

Para aceitar decimais e negativos, a forma completa usa `try/except` dentro do `while` (capítulo 12 do guia).

Até teclar *Enter*

sentinela.py

```
compras = []
item = input("Item (ENTER para terminar): ").strip()
while item:                                # texto vazio é False
    compras.append(item)
    item = input("Item (ENTER para terminar): ").strip()
print(f"{len(compras)} itens:", ", ", ".join(compras))
```

SAÍDA (PÃO, ARROZ, LEITE E ENTER)

```
Item (ENTER para terminar): pão
Item (ENTER para terminar): arroz
Item (ENTER para terminar): leite
Item (ENTER para terminar):
3 itens: pão, arroz, leite
```

- ✓ **Sentinela:** um valor especial que avisa "acabou" (aqui, o texto vazio)
- ✓ `while item:` usa o "texto vazio é falso" da aula 13
- ✓ Pergunta antes do laço e de novo no fim de cada volta

É a lista de compras da aula 09, agora aceitando quantos itens o usuário quiser.

Laço "infinito" *com saída*

while_true.py

```
while True:
    senha = input("Senha: ")
    if senha == "python123":
        print("Acesso liberado")
        break
    print("Senha errada, tente de novo")
```

SAÍDA (ABC, 123, PYTHON123)

```
Senha: abc
Senha errada, tente de novo
Senha: 123
Senha errada, tente de novo
Senha: python123
Acesso liberado
```

- ✓ `while True` nunca para sozinho: a saída é o `break`
- ✓ Evita repetir o `input()` antes e dentro do laço
- ✓ Padrão comum em menus (exercício da seção 07)

Laço infinito sem querer? `Ctrl` + `C` no terminal interrompe o programa (`KeyboardInterrupt`).

O else *do laço*

tentativas.py

```
SEGREDO = 7
tentativas = 0
while tentativas < 3:
    palpite = int(input("Palpite (1 a 10): "))
    tentativas += 1
    if palpite == SEGREDO:
        print(f"Acertou em {tentativas} tentativa(s)!")
        break
    print("Maior" if palpite < SEGREDO else "Menor")
else:
    print("Acabaram as tentativas. Era", SEGREDO)
```

SAÍDA (PALPITES 5, 9 E 6)

```
Palpite (1 a 10): 5
Maior
Palpite (1 a 10): 9
Menor
Palpite (1 a 10): 6
Maior
Acabaram as tentativas. Era 7
```

- ✓ O `else` do `while` roda quando o laço termina **sem** `break`
- ✓ Aqui: acabaram as 3 tentativas sem acertar
- ✓ Se acertar, o `break` pula o `else`
- ✓ O `for` também tem `else`, com a mesma regra

Pouco conhecido e às vezes confuso: use com um comentário.
Uma variável `acertou = False` faz o mesmo de forma explícita.

Até atingir *um objetivo*

juros.py

```
saldo = 1000.0
meses = 0
while saldo < 2000:
    saldo *= 1.01      # 1% ao mês
    meses += 1
print(f"{meses} meses para dobrar: R$ {saldo:.2f}")
```

70 meses para dobrar: R\$ 2006.76

- ✓ Não se sabe quantos meses serão: perfeito para `while`
- ✓ A condição é o objetivo (`saldo < 2000`) ao contrário: repete enquanto não chegou
- ✓ A 1% ao mês, o dinheiro dobra em **70 meses**

USE FOR QUANDO	USE WHILE QUANDO
Percorre uma coleção	Depende do usuário
Sabe quantas voltas	Repete até uma condição mudar

Um *menu* que repete

Enunciado: um menu com depositar, ver saldo e sair, que repete até a pessoa escolher 0 e avisa quando a opção não existe.

Uma resposta possível

```
# Exercício: menu que repete até sair
saldo = 0
while True:
    opcao = input("[1] Depositar [2] Ver saldo [0] Sair: ")
    if opcao == "1":
        saldo += float(input("Valor: "))
    elif opcao == "2":
        print(f"Saldo: R$ {saldo:.2f}")
    elif opcao == "0":
        print("Até logo!")
        break
    else:
        print("Opção inválida")
```

SAÍDA (DEPOSITA 150 E 49.9, VÊ O SALDO, TENTA 7, SAI)

```
[1] Depositar [2] Ver saldo [0] Sair: 1
Valor: 150
[1] Depositar [2] Ver saldo [0] Sair: 1
Valor: 49.9
[1] Depositar [2] Ver saldo [0] Sair: 2
Saldo: R$ 199.90
[1] Depositar [2] Ver saldo [0] Sair: 7
Opção inválida
[1] Depositar [2] Ver saldo [0] Sair: 0
Até logo!
```

- ✓ `while True` + `break` na opção 0
- ✓ `if / elif / else` (aula 13) escolhem a ação
- ✓ Desafio: incluía sacar, sem deixar o saldo ficar negativo

Teste o que *ficou*

1. Quando o while para?

A) Depois de 10 voltas

B) Quando a condição fica falsa (ou num break)

C) Nunca

D) No fim do arquivo

B.

2. Programa preso num laço infinito. O que fazer?

A) Ctrl + C no terminal

B) Ctrl + S

C) Esperar

D) Ctrl + Z no editor

A. Interrompe com KeyboardInterrupt.

3. Quando roda o else de um while?

A) Sempre

B) Quando termina sem break

C) Quando há break

D) Nunca

B.

4. "Peça a idade até vir um número" pede:

A) for com range

B) while

C) if

D) match

B. Não se sabe quantas tentativas.

Repetir até *dar certo*

O `while` repete enquanto a condição é verdadeira e precisa de algo que a mude. Serve para validar entradas, ler até uma sentinela, menus com `while True` e `break`, e simulações até um objetivo. O `else` do laço roda quando não houve `break`.

`while condição`

Mude a condição

Sentinela

`while True + break`

`Ctrl + C`

Próxima aula → 16 · Utilizando List Comprehension

Fontes: docs.python.org/3/reference/compound_stmts.html#the-while-statement; docs.python.org/3/tutorial/controlflow.html (`break`, `continue`, `else clauses on loops`), consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.